This notebook will guide you on how to take the files directly downloaded from OCDb and prep them to use in various modeling frameworks in the next notebooks.

Use the workshop environment with PICASO, Virga, Astropy, NumPy, pandas, and Matplotlib installed. We won't need PICASO yet for this notebook, but will in the following notebooks. Much of this notebook derives from the "Getting Started" Virga tutorial.

First, we load all our necessary packages and imports

Code cell 2

```
import os
from pathlib import Path
import pandas as pd
import glob
import virga.justdoit as jdi
import virga.justplotit as jpi
import virga.ior_factory as ior
from bokeh.io import output_notebook
output_notebook()
import matplotlib.pyplot as plt
import numpy as np
import matplotlib.pyplot as plt

workshop_dir = Path.cwd()
if not (workshop_dir / "data").exists():
    workshop_dir = workshop_dir.parent
```

Interactive plot: open the solution notebook to explore this figure.

Next, we're going to load and parse the OCDb format into something our modeling tools can use (this will depend on your tool of choice!). Our tool of choice here is the PICASO-Virga framework, so that dictates our format.

We're going to convert two different gas tholin ("haze") datasets to use for our next notebook: 1) Drant et al., 2026 which is a haze derived from a room temperature experiment with N2/CH4/CO gas at 95:4.95:0.05 ratios

and

2) He et al. 2024, which is a haze derived from a 400 K experiment with H_2O:CH_4:N_2:CO_2:He gas at 66.0:6.6:6.5:4.9:16.0 ratios

Code cell 4

```
path = str(workshop_dir / 'data' / 'ocdb') + os.sep
file_drant = 'Drant_2026_A&A_dataset-3_gastholin_N2-CH4-CO_95-4.95-0.05_298K_all.csv'
file_he =
'He_2024_NatureAstronomy_dataset-2_gastholin_H2O-CH4-N2-CO2-He_66.0-6.6-6.5-4.9-16.0_400K_all.csv'

# Load your datasets
col_names = ['wave','n','k']
drant =
pd.read_csv(path+file_drant,skiprows=19,names=col_names,usecols=[0,1,4],sep=r'\s+',encoding='utf-16')
he =
pd.read_csv(path+file_he,skiprows=26,names=col_names,usecols=[0,1,4],sep=r'\s+',encoding='utf-16')
```

```
# Lab data tends to be *very* high resolution compared to observations,
# so usually we bin down the dataset to a lower resolution, which speeds up model runtimes

skip = 20 # let's grab every 20th step in the arrays
# -- this is a little lower than "science-ready", but will be fast for our demos!

# Make them into arrays so numpy is happy
wvr,real,imag =
np.array(drant['wave'][::skip]),np.array(drant['n'][::skip]),np.array(drant['k'][::skip])

# # Virga expects an index in the file so let's add it!
index = np.arange(1,len(wvr)+1,1)


# Write to a file to grab in our next demo. Virga expects a "refrind" file extension to do the next
step.
with open(workshop_dir / 'data' / 'processed' / 'Drant_N2_CH4_CO_95_4.95_0.05_tholin.refrind', 'w') as
 f:
     for i,a,b,c in zip(index, wvr, real, imag):
            print("%d %.9f %.9f %.9f" % (i, a, b, c), file = f)


# Now we'll do the He data, which is even higher resolution and needs even fewer points for speed

skip = 100
wvr_he,real_he,imag_he =
np.array(he['wave'][::skip]),np.array(he['n'][::skip]),np.array(he['k'][::skip])

index = np.arange(1,len(wvr_he)+1,1)


# Write to a file to grab in our next demo. Virga expects a "refrind" file extension to do the next
step.
with open(workshop_dir / 'data' / 'processed' / 'He_400K_Water-rich_tholin.refrind', 'w') as f:
     for i,a,b,c in zip(index, wvr_he, real_he, imag_he):
            print("%d %.9f %.9f %.9f" % (i, a, b, c), file = f)
```

Now let's plot our new and rebinned data to make sure it looks good!

Code cell 6

```
plt.figure(figsize=(12, 9))
plt.rcParams['axes.labelsize'] = 22 # X and Y label size
plt.rcParams['xtick.labelsize'] = 20 # X tick label size
plt.rcParams['ytick.labelsize'] = 20 # Y tick label size

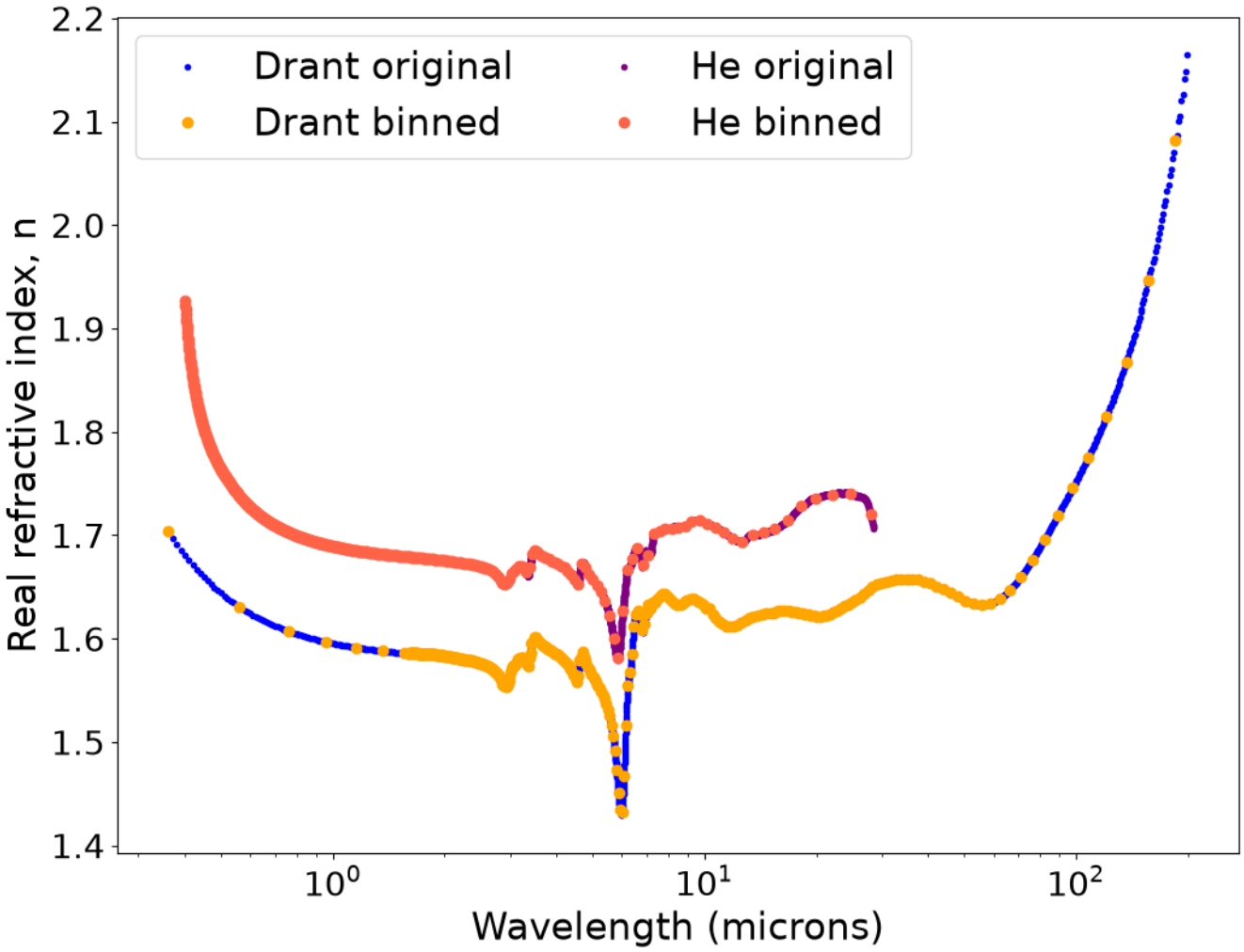

plt.errorbar(np.array(drant['wave']),np.array(drant['n']),fmt='.',color='blue',label="Drant original")
plt.errorbar(wvr,real,fmt='.',color='orange',markersize=12,label='Drant binned')

# let's apply an offset to the He data so we can visualize it better
offset=0.1
plt.errorbar(np.array(he['wave']),np.array(he['n'])+offset,fmt='.',color='purple',label="He original")
plt.errorbar(wvr_he,real_he+offset,fmt='.',color='tomato',markersize=12,label='He binned')


plt.legend(fontsize=22,ncol=2)
plt.xscale('log')
plt.ylabel('Real refractive index, n')
```

```
plt.xlabel('Wavelength (microns)')
```

```
Text(0.5, 0, 'Wavelength (microns)')
```



The binned data is sitting exactly on the originals, great! Now we're going to run a Mie code on these refractive indices to turn them into Mie coefficients, which many radiative transfer models use.

Note that Mie theory inherently assumes spherical particles, but this is not always the case for clouds, hazes, or ices! That's another Webbinar though. For now, let's just do Mie theory.

The next notebook will also do this step, but we've broken it out more explicitly here to see the process generically. This step uses Virga for the first time. We're only calculating the Mie coefficients for a few radii here for speed, but experiment with multiple options and see how that changes your runtime.

Note that the upper and lower limits of the radii are given here in centimeters!

We start with 10 radii, but try 5, 20, and ultimately 40 for use in following notebooks.

Code cell 8

```
refrind_dir = str(workshop_dir / 'data' / 'processed') # <---- this is where you've sent your
formatted file
import glob
out_dir = refrind_dir
```

```
from pathlib import Path


directory = Path(refrind_dir)
for file in directory.glob('*.refrind'):
    file_name_var = file.stem
    gas_name = file_name_var

    qext, qscat, g_qscat, radii,wave = jdi.calc_mie_db([gas_name],
                                refrind_dir, out_dir, rmin = 1e-5,rmax=1e-3,nradii
=10)#,PyMie=True,auto=False)
#print(radii)
```

```
Computing optical properties for Drant_N2_CH4_CO_95_4.95_0.05_tholin using MiePython...
If this function seems to be running a really long time... Check that you gave your rmin (and r_mon if
 using aggregates) in centimeters and that you are not making tennis balls :)

360 wavelengths of refractive index data found for Drant_N2_CH4_CO_95_4.95_0.05_tholin. Grid of mean
radii (in bins) to calculate extinction and scattering efficiencies for (in cm):

           min             mean             max          bin width (dr)
       7.495969e-06    1.000000e-05    1.250403e-05       5.008061e-06
       1.250403e-05    1.668101e-05    2.085798e-05       8.353950e-06
       2.085798e-05    2.782559e-05    3.479321e-05       1.393523e-05
       3.479321e-05    4.641589e-05    5.803857e-05       2.324536e-05
       5.803857e-05    7.742637e-05    9.681417e-05       3.877560e-05
       9.681417e-05    1.291550e-04    1.614958e-04       6.468160e-05
       1.614958e-04    2.154435e-04    2.693912e-04       1.078954e-04
       2.693912e-04    3.593814e-04    4.493716e-04       1.799804e-04
       4.493716e-04    5.994843e-04    7.495969e-04       3.002254e-04
       7.495969e-04    1.000000e-03    1.250403e-03       5.008061e-04

Averages from 6 sub-bins will be used to calculate the properties that represent the mean radius in
each bin above.
Optical properties for Drant_N2_CH4_CO_95_4.95_0.05_tholin have been calculated and saved as /Users/jj
m6243/Documents/jwebbinar_prep/jwebbinar53/data/processed/Drant_N2_CH4_CO_95_4.95_0.05_tholin.mieff.


Computing optical properties for He_400K_Water-rich_tholin using MiePython...
If this function seems to be running a really long time... Check that you gave your rmin (and r_mon if
 using aggregates) in centimeters and that you are not making tennis balls :)

512 wavelengths of refractive index data found for He_400K_Water-rich_tholin. Grid of mean radii (in
bins) to calculate extinction and scattering efficiencies for (in cm):

           min             mean             max          bin width (dr)
       7.495969e-06    1.000000e-05    1.250403e-05       5.008061e-06
       1.250403e-05    1.668101e-05    2.085798e-05       8.353950e-06
       2.085798e-05    2.782559e-05    3.479321e-05       1.393523e-05
       3.479321e-05    4.641589e-05    5.803857e-05       2.324536e-05
       5.803857e-05    7.742637e-05    9.681417e-05       3.877560e-05
       9.681417e-05    1.291550e-04    1.614958e-04       6.468160e-05
       1.614958e-04    2.154435e-04    2.693912e-04       1.078954e-04
       2.693912e-04    3.593814e-04    4.493716e-04       1.799804e-04
       4.493716e-04    5.994843e-04    7.495969e-04       3.002254e-04
       7.495969e-04    1.000000e-03    1.250403e-03       5.008061e-04

Averages from 6 sub-bins will be used to calculate the properties that represent the mean radius in
each bin above.
Optical properties for He_400K_Water-rich_tholin have been calculated and saved as
/Users/jjm6243/Documents/jwebbinar_prep/jwebbinar53/data/processed/He_400K_Water-rich_tholin.mieff.
```

Finally, let's examine these Mie coefficients we've just calculated to see how they change as a function of radius.

Mie coefficients come in three flavors: scattering, extinction (which is just scattering + absorption), and scattering asymmetry parameter. Technically we output efficiences, Q, rather than coefficients. Let's look at two parameters:

Asymmetry (g_qscat/qscat) : will show how forward/backscattering the gas is

Qscat / Qext (qscat/qext) : will show you how bright the condensate is expected to be

First, let's look at the ratio of scattering to extinction:

Code cell 10

```
plt.figure(figsize=(15, 9))
plt.rcParams['axes.labelsize'] = 22 # X and Y label size
plt.rcParams['xtick.labelsize'] = 20 # X tick label size
plt.rcParams['ytick.labelsize'] = 20 # Y tick label size



tholin = 'Drant_N2_CH4_CO_95_4.95_0.05_tholin'
for i in range(0,len(radii),1):
    folder = refrind_dir
    qext, qscat, g_qscat, nwave,radii,wave = jdi.get_mie(tholin,folder)
    plt.plot(wave[:,i], qscat[:,i]/qext[:,i], color='blue')
    if i==0:
        plt.plot(wave[:,i], qscat[:,i]/qext[:,i], color='blue',label='Drant data')

tholin = 'He_400K_Water-rich_tholin'
for i in range(0,len(radii),1):
    folder = refrind_dir
    qext, qscat, g_qscat, nwave,radii,wave = jdi.get_mie(tholin,folder)
    plt.plot(wave[:,i], qscat[:,i]/qext[:,i], color='orange')
    if i==0:
        plt.plot(wave[:,i], qscat[:,i]/qext[:,i], color='orange',label='He data')

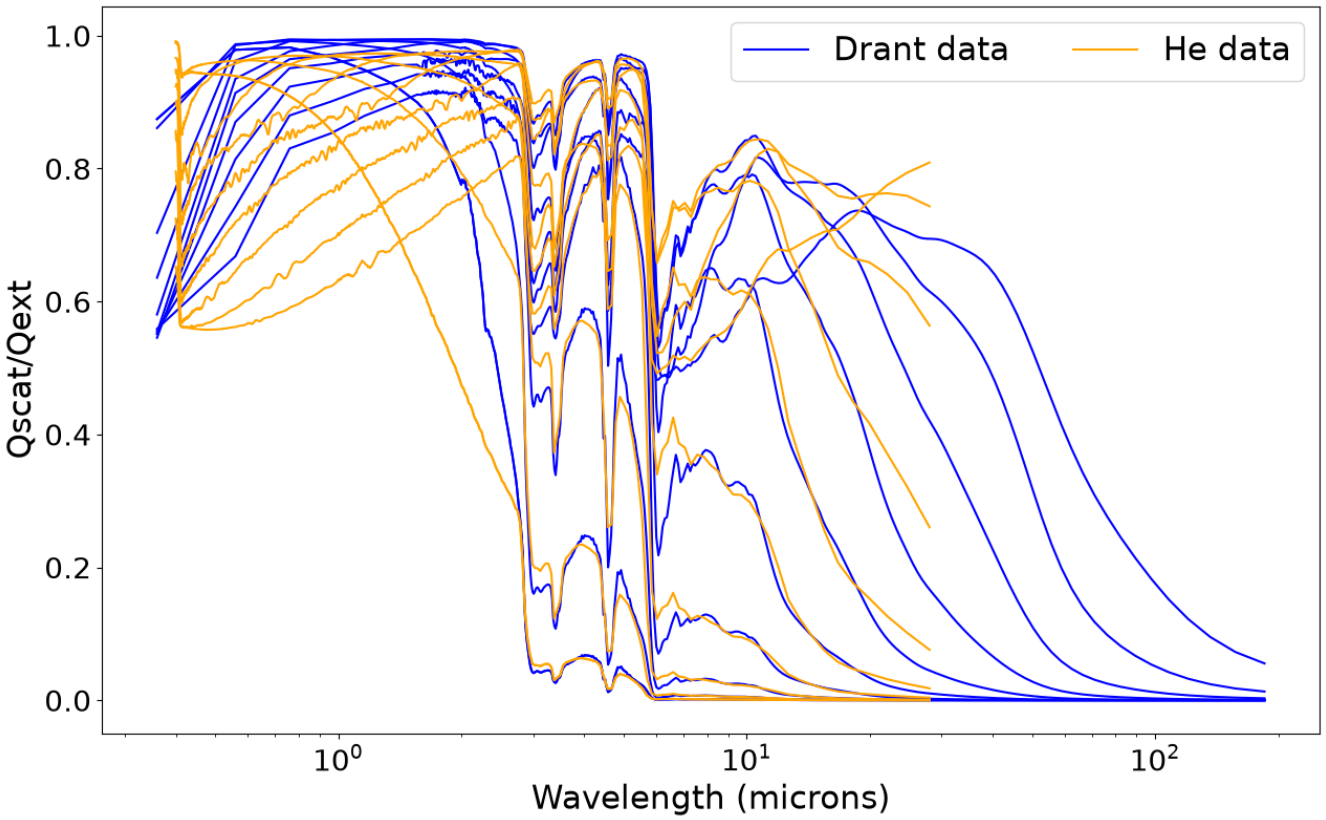



plt.legend(loc=1,fontsize=22,ncol=2)
plt.xscale('log')
plt.ylabel('Qscat/Qext')
plt.xlabel('Wavelength (microns)')



```

```
Text(0.5, 0, 'Wavelength (microns)')
```



Now let's look at the asymmetry parameter:

Code cell 12

```
plt.figure(figsize=(15, 9))
plt.rcParams['axes.labelsize'] = 22 # X and Y label size
plt.rcParams['xtick.labelsize'] = 20 # X tick label size
plt.rcParams['ytick.labelsize'] = 20 # Y tick label size



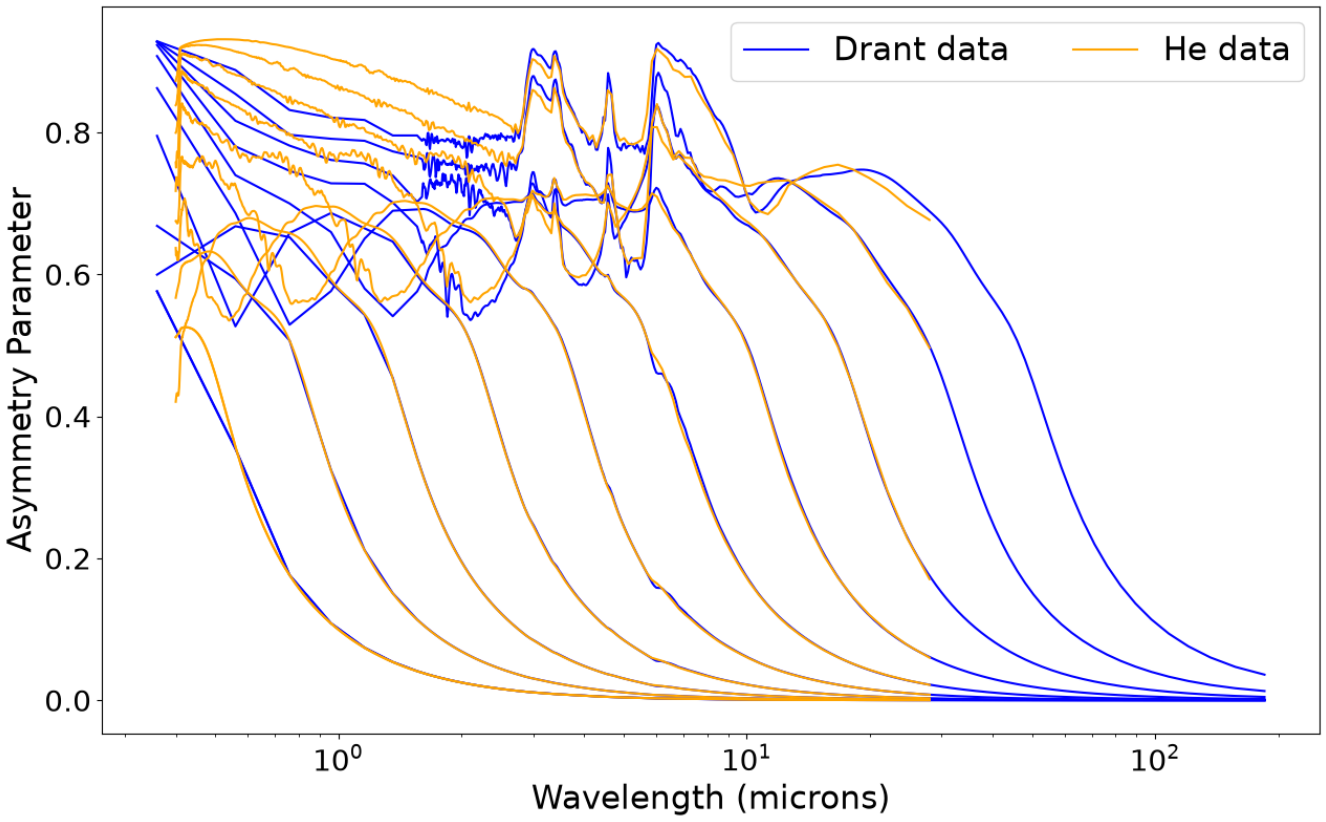
tholin = 'Drant_N2_CH4_CO_95_4.95_0.05_tholin'
for i in range(0,len(radii),1):
    folder = refrind_dir
    qext, qscat, g_qscat, nwave,radii,wave = jdi.get_mie(tholin,folder)
    plt.plot(wave[:,i], g_qscat[:,i]/qscat[:,i], color='blue')
    if i==0:
        plt.plot(wave[:,i], g_qscat[:,i]/qscat[:,i], color='blue',label='Drant data')

tholin = 'He_400K_Water-rich_tholin'
for i in range(0,len(radii),1):
    folder = refrind_dir
    qext, qscat, g_qscat, nwave,radii,wave = jdi.get_mie(tholin,folder)
    plt.plot(wave[:,i], g_qscat[:,i]/qscat[:,i], color='orange')
    if i==0:
        plt.plot(wave[:,i], g_qscat[:,i]/qscat[:,i], color='orange',label='He data')



plt.legend(loc=1,fontsize=22,ncol=2)
plt.xscale('log')
plt.ylabel('Asymmetry Parameter')
plt.xlabel('Wavelength (microns)')
```

```
Text(0.5, 0, 'Wavelength (microns)')
```



In both cases, you can see how strange things happen at the edges of your wavelength range, as well as where we have over-binned the original data. It's always important to consider carefully your conversion from laboratory data into model data product!

Now it's time to move to Notebook 2 and examine water ice in the context of Brown Dwarf models.